

SEQUENTIAL LOGIC DESIGN

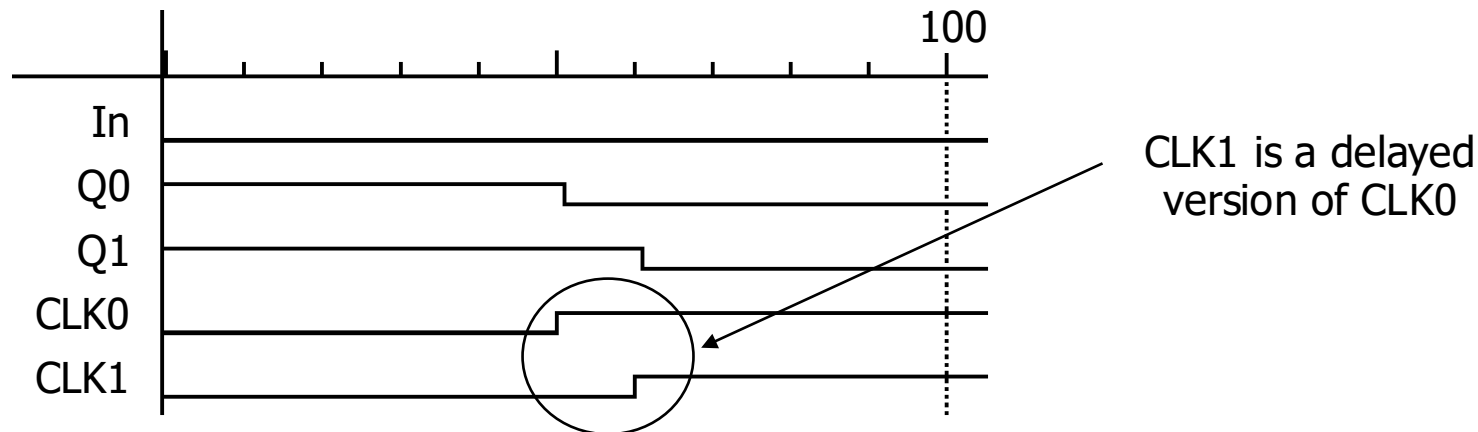
Module.3

CDA 3103C

Mesut Ozdag, Ph.D.

Clock Skew

- Clock skew is the difference in time between when two state elements see a clock edge.
- If the clock skew is large enough, it may be possible for a state element to change and cause the input to another flip-flop to change before the clock edge is seen by the second flip-flop.
 - To avoid incorrect operation, we must increase the clock period to allow for the maximum clock skew.



original state: $IN = 0$, $Q0 = 1$, $Q1 = 1$

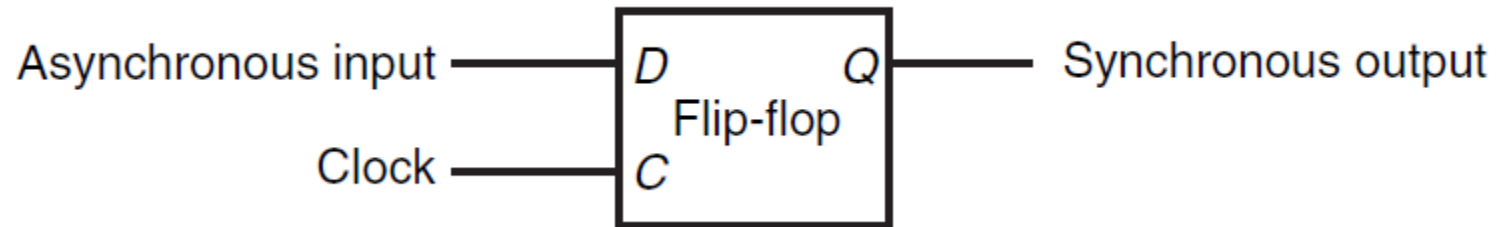
due to skew, next state becomes: $Q0 = 0$, $Q1 = 0$, and not $Q0 = 0$, $Q1 = 1$

Asynchronous Inputs

- Clocked synchronous circuits
 - Inputs, state, and outputs sampled or changed in relation to a common reference signal (called the clock)
- Asynchronous circuits
 - Inputs, state, and outputs sampled or changed independently of a common reference signal
 - Some system inputs must be asynchronous
 - reset signal, memory wait, user input
 - Unstable states are a major concern
- Asynchronous inputs to synchronous circuits
 - Because synchronous inputs are greatly preferred, we use a synchronizer

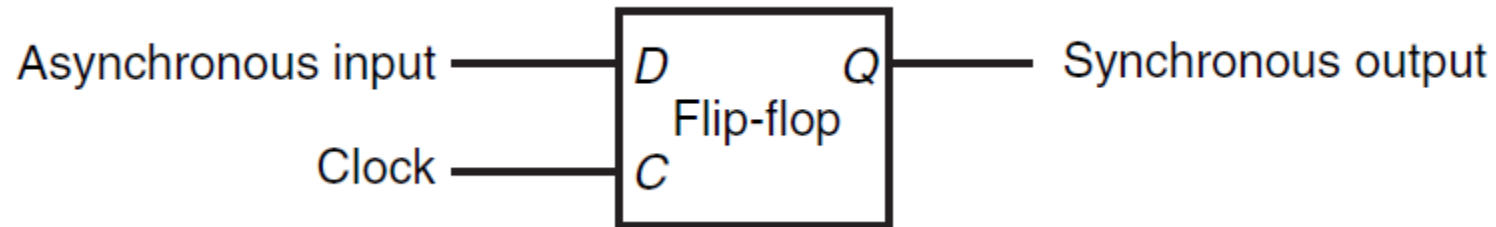
Synchronizer

- Use a synchronizer to translate asynchronous input

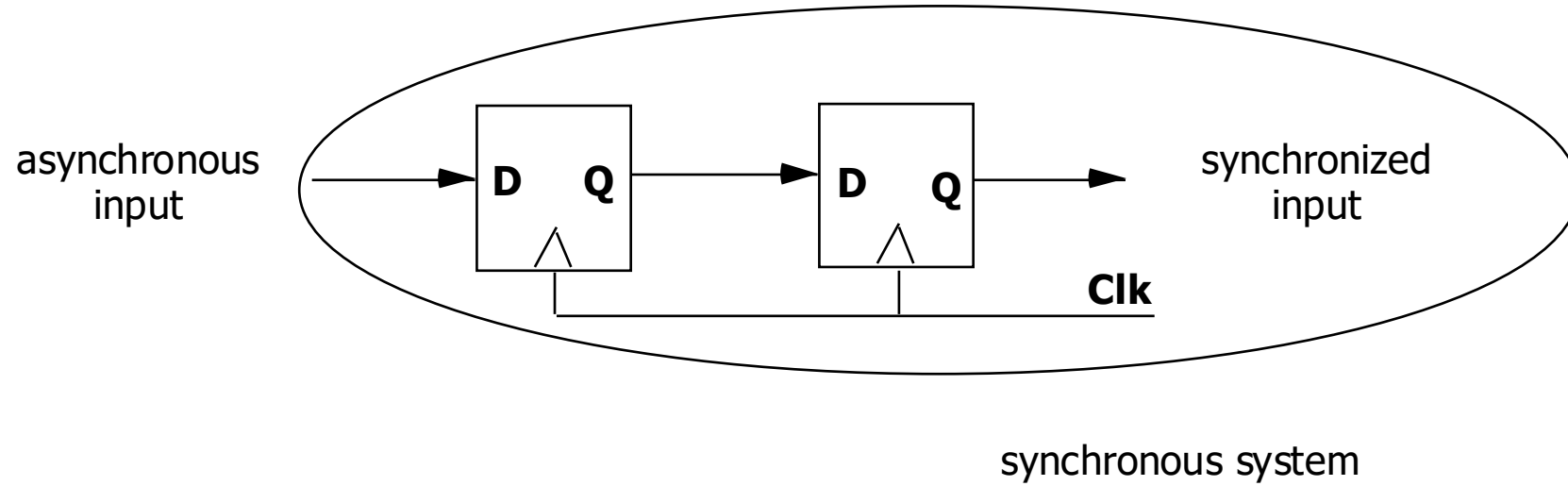


Synchronizer

- What happens if the asynchronous input is transitioning between 0 and 1?
 - The signal is not stable for the required setup and hold times
 - The synchronizer may enter a metastable state
 - The output will be neither 0 or 1
 - This situation is called synchronizer failure



Synchronous System



Sequential Hardware Elements

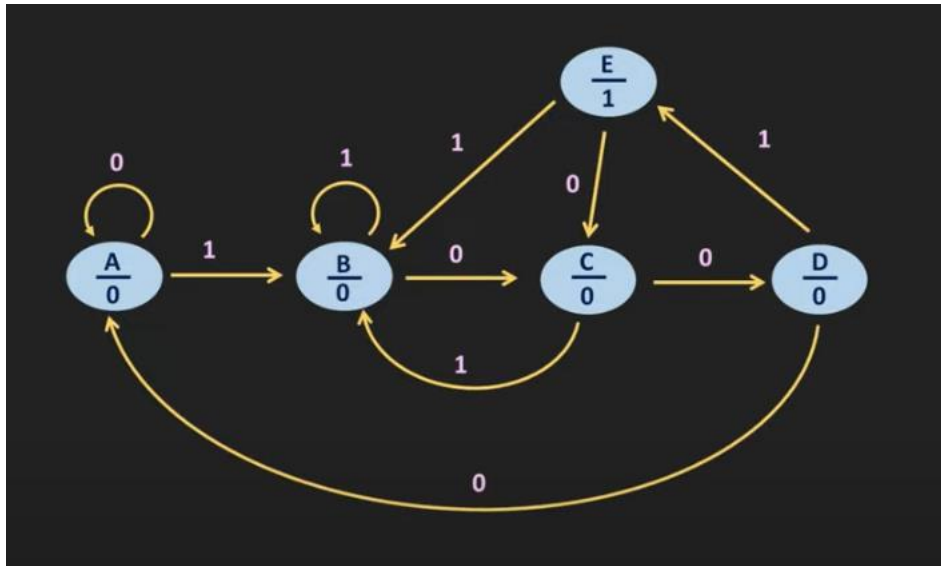
- Collections of flip-flops with similar controls and logic
 - Values in individual flip-flops are related
 - Shared clock signal
 - Similar logic at each stage
- Examples
 - Pattern Recognizer
 - Counter
 - Shift register
 - Data Register
 - Register File

Pattern Recognizer

- Combinational function of input samples
 - In this case, recognizing the pattern 1001 on the single input signal
 - X= 00100100100 (input)
 - Y= 00000100100 (overlapping) (output)
 - Z= 00000100000 (non-overlapping) (output)

Pattern Recognizer

1001 overlapping pattern recognizer:



PS	Next State		Output
	X = 0	X = 1	
A	A	B	0
B	C	B	0
C	D	B	0
D	A	E	0
E	C	B	1

A – 000

B – 001

C – 010

E – 011

D – 100

101, 110, 111 - Don't Care

1001 overlapping pattern recognizer:

Draw a State Table

PS	Next State		Output
	X = 0	X = 1	
000	000	001	0
001	010	001	0
010	100	001	0
100	000	011	0
011	010	001	1

D Flip-Flop



Input	PS			NS			Output
	Q2	Q1	Q0	Q2+	Q1+	Q0+	
X							Y
0	0	0	0	0	0	0	0
0	0	0	1	0	1	0	0
0	0	1	0	1	0	0	0
0	0	1	1	0	1	0	1
0	1	0	0	0	0	0	0
1	0	0	0	0	0	1	0
1	0	0	1	0	0	1	0
1	0	1	0	0	0	1	0
1	0	1	1	0	0	1	1
1	1	0	0	0	1	1	0

Find the Boolean Expression

Input	PS			NS			Inputs to FF			Output
	Q2	Q1	Q0	Q2+	Q1+	Q0+	D2	D1	D0	
X										Y
0	0	0	0	0	0	0	0	0	0	0
0	0	0	1	0	1	0	0	1	0	0
0	0	1	0	1	0	0	1	0	0	0
0	0	1	1	0	1	0	0	1	0	1
0	1	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	1	0	0	1	0
1	0	0	1	0	0	1	0	0	1	0
1	0	1	0	0	0	1	0	0	1	0
1	0	1	1	0	0	1	0	0	1	1
1	1	0	0	0	1	1	0	1	1	0

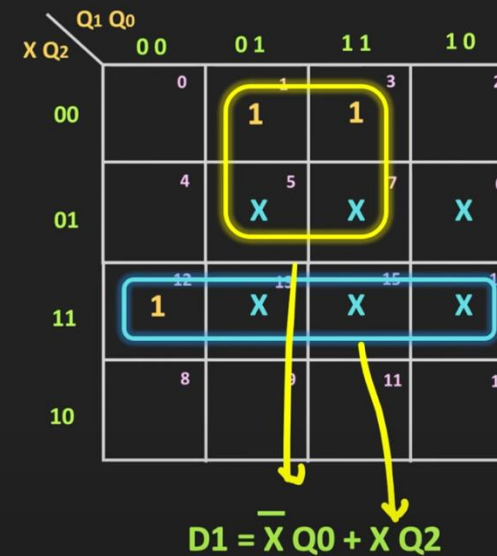
X Q2 \ Q1 Q0	0 0	0 1	1 1	1 0
0 0	0	1	3	2
0 1	4	X	X	6
1 1	12	X	X	14
1 0	8	9	11	10

$$D2 = \overline{X} \overline{Q1} \overline{Q0}$$

Minterms - m5, m6, m7, m13, m14, m15 are don't care terms

Find the Boolean Expression

Input	PS			NS			Inputs to FF			Output
	Q2	Q1	Q0	Q2+	Q1+	Q0+	D2	D1	D0	
X										Y
0	0	0	0	0	0	0	0	0	0	0
0	0	0	1	0	1	0	0	1	0	0
0	0	1	0	1	0	0	1	0	0	0
0	0	1	1	0	1	0	0	1	0	1
0	1	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	1	0	0	1	0
1	0	0	1	0	0	1	0	0	1	0
1	0	1	0	0	0	1	0	0	1	0
1	0	1	1	0	0	1	0	0	1	1
1	1	0	0	0	1	1	0	1	1	0



Minterms - m5, m6, m7, m13, m14, m15 are don't care terms

Find the Boolean Expression

Input	PS			NS			Inputs to FF			Output
	Q2	Q1	Q0	Q2+	Q1+	Q0+	D2	D1	D0	Y
X										
0	0	0	0	0	0	0	0	0	0	0
0	0	0	1	0	1	0	0	1	0	0
0	0	1	0	1	0	0	1	0	0	0
0	0	1	1	0	1	0	0	1	0	1
0	1	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	1	0	0	1	0
1	0	0	1	0	0	1	0	0	1	0
1	0	1	0	0	0	1	0	0	1	0
1	0	1	1	0	0	1	0	0	1	1
1	1	0	0	0	1	1	0	1	1	0

		Q1 Q0			
		00	01	11	10
X Q2	00	0	1	3	2
	01	4	X	X	X
	11	12	13	15	14
	10	8	9	11	10

D0 = X

Minterms - m5, m6, m7, m13, m14, m15 are don't care terms

Find the Boolean Expression

Input	PS			NS			Inputs to FF			Output
X	Q2	Q1	Q0	Q2+	Q1+	Q0+	D2	D1	D0	Y
0	0	0	0	0	0	0	0	0	0	0
0	0	0	1	0	1	0	0	1	0	0
0	0	1	0	1	0	0	1	0	0	0
0	0	1	1	0	1	0	0	1	0	1
0	1	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	1	0	0	1	0
1	0	0	1	0	0	1	0	0	1	0
1	0	1	0	0	0	1	0	0	1	0
1	0	1	1	0	0	1	0	0	1	1
1	1	0	0	0	1	1	0	1	1	0

X Q ₂	Q ₁ Q ₀			
	0 0	0 1	1 1	1 0
0 0	0	1	3 1	2
0 1	4	5 X	7 X	6 X
1 1	12	13 X	15 X	14 X
1 0	8	9	11 1	10

$$Y = Q_1 Q_0$$

Minterms - m5, m6, m7, m13, m14, m15 are don't care terms

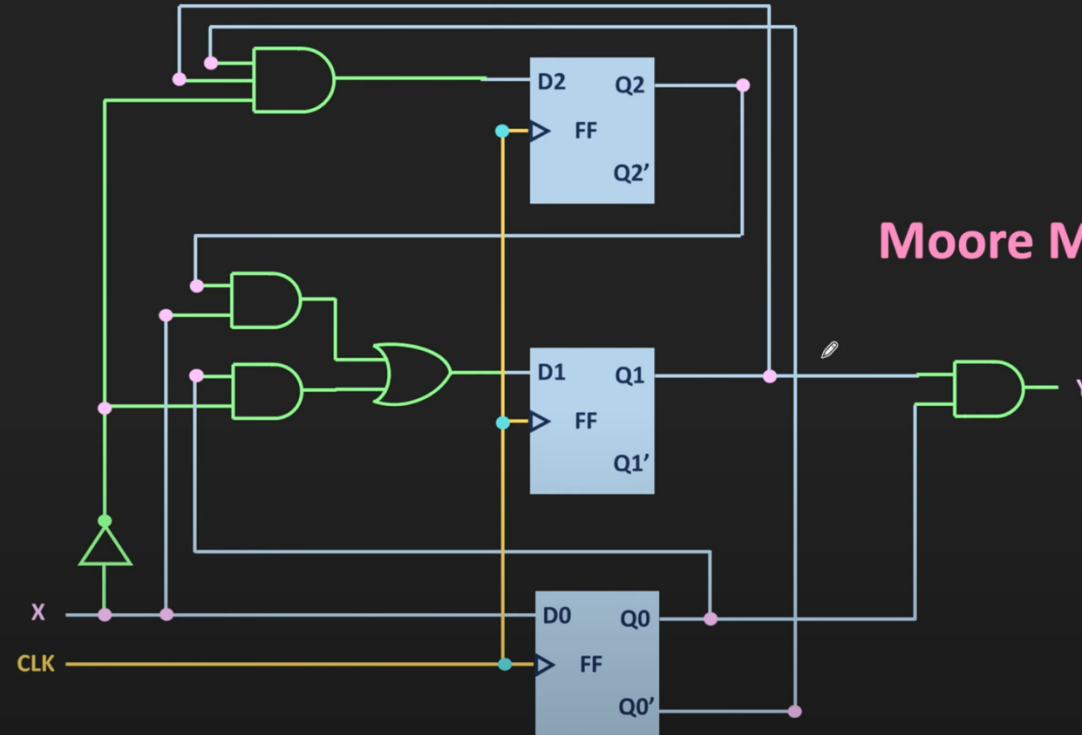
Draw Logic Circuit

$$Y = Q_1 Q_0$$

$$D_2 = \overline{X} Q_1 \overline{Q_0}$$

$$D_1 = \overline{X} Q_0 + X Q_2$$

$$D_0 = X$$



Moore Machine

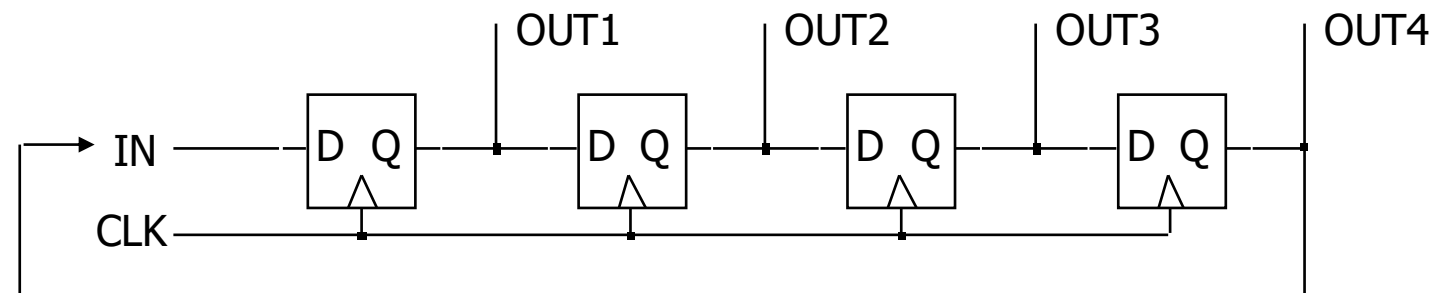
Pattern Recognizer

- 1001 non-overlapping?

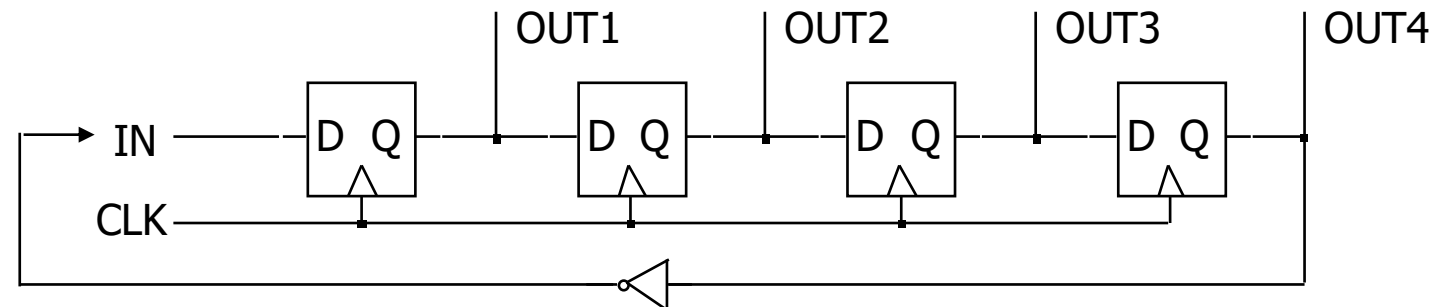
<https://www.youtube.com/watch?v=ZCRrECZe7YA>

Counters

- Sequences through a fixed set of patterns
 - In this case, 1000, 0100, 0010, 0001
 - If one of the patterns is its initial state (by loading or set/reset)

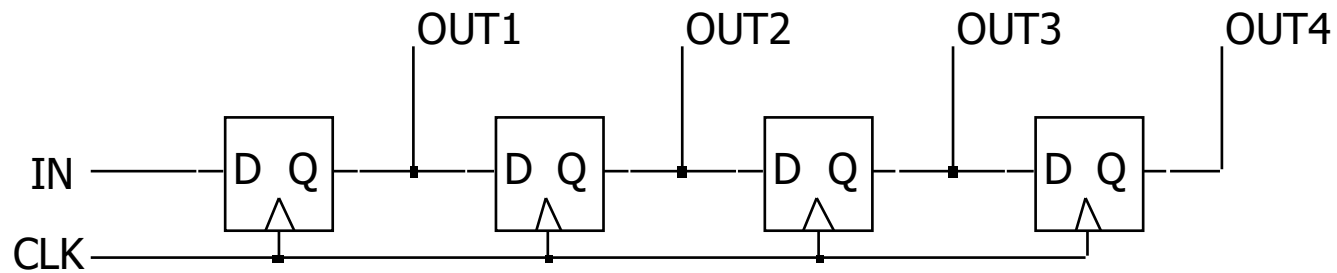


- Mobius (or Johnson) counter
 - In this case, 1000, 1100, 1110, 1111, 0111, 0011, 0001, 0000



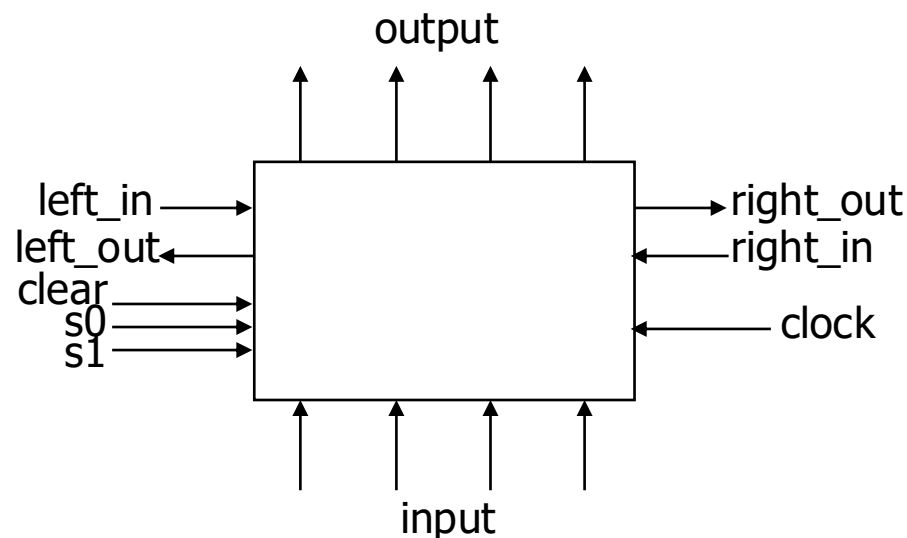
Shift Register

- Holds samples of input
 - Store last 4 input values in sequence
 - 4-bit shift register:



Universal Shift Register

- Holds 4 values
 - Serial or parallel inputs
 - Serial or parallel outputs
 - Permits shift left or right
 - Shift in new values from left or right



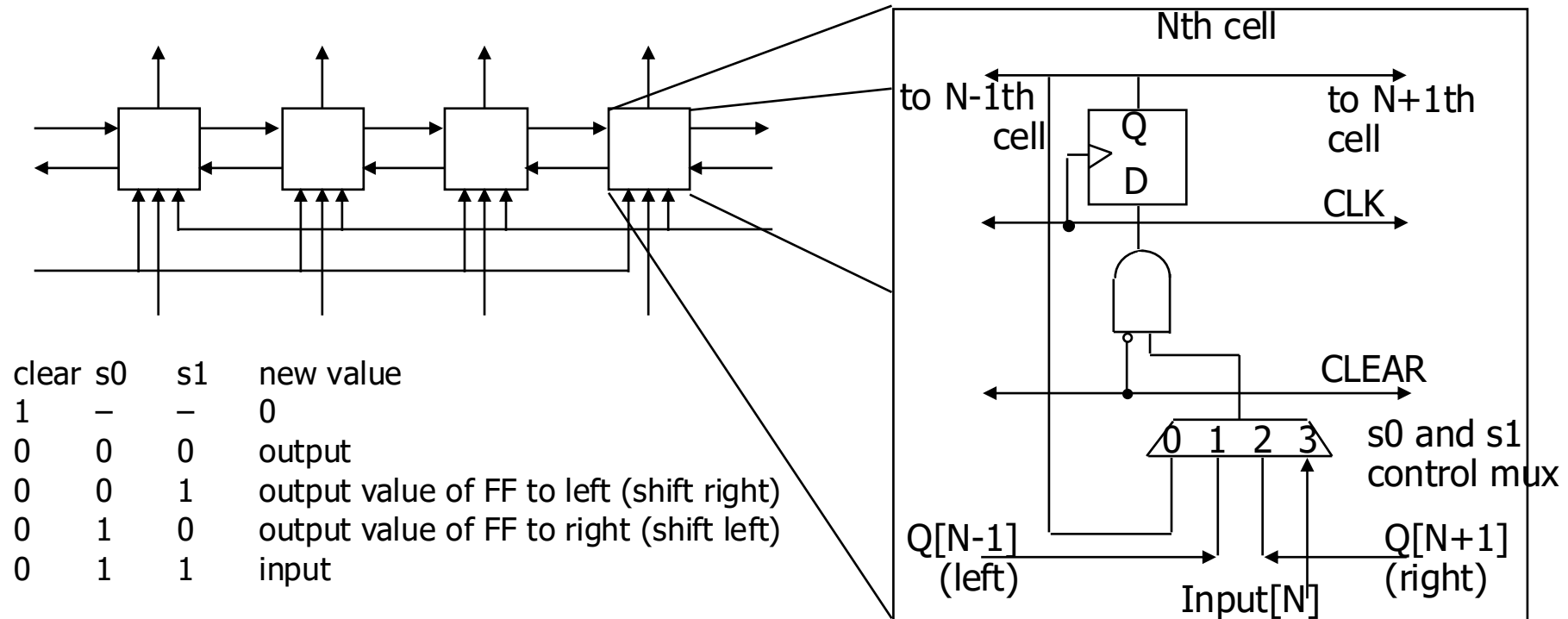
clear sets the register contents and output to 0

s1 and s0 determine the shift function

s0	s1	function
0	0	hold state
0	1	shift right
1	0	shift left
1	1	load new input

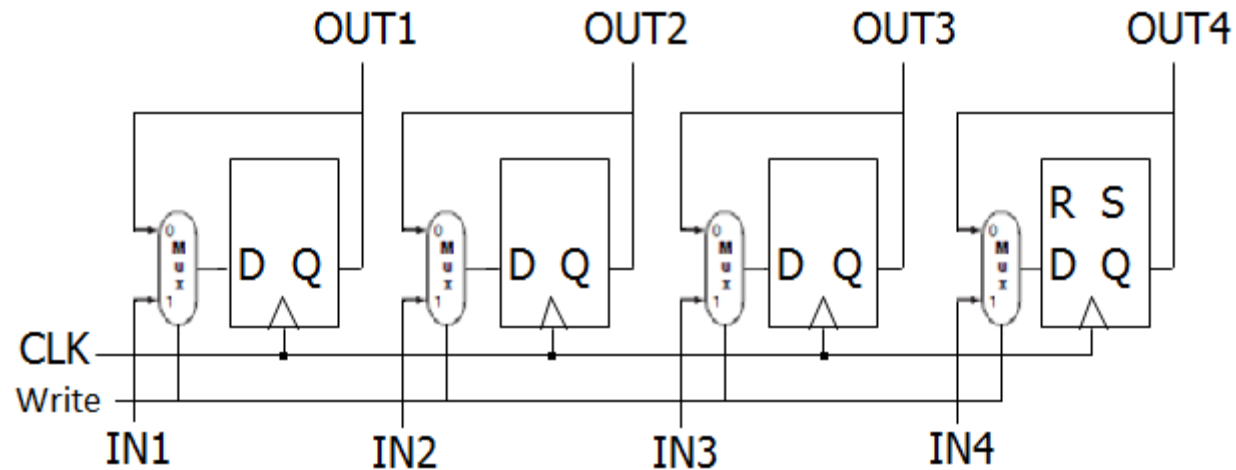
Design of Universal Shift Register

- Consider one of the four flip-flops
 - New value at next clock cycle:



Data Register

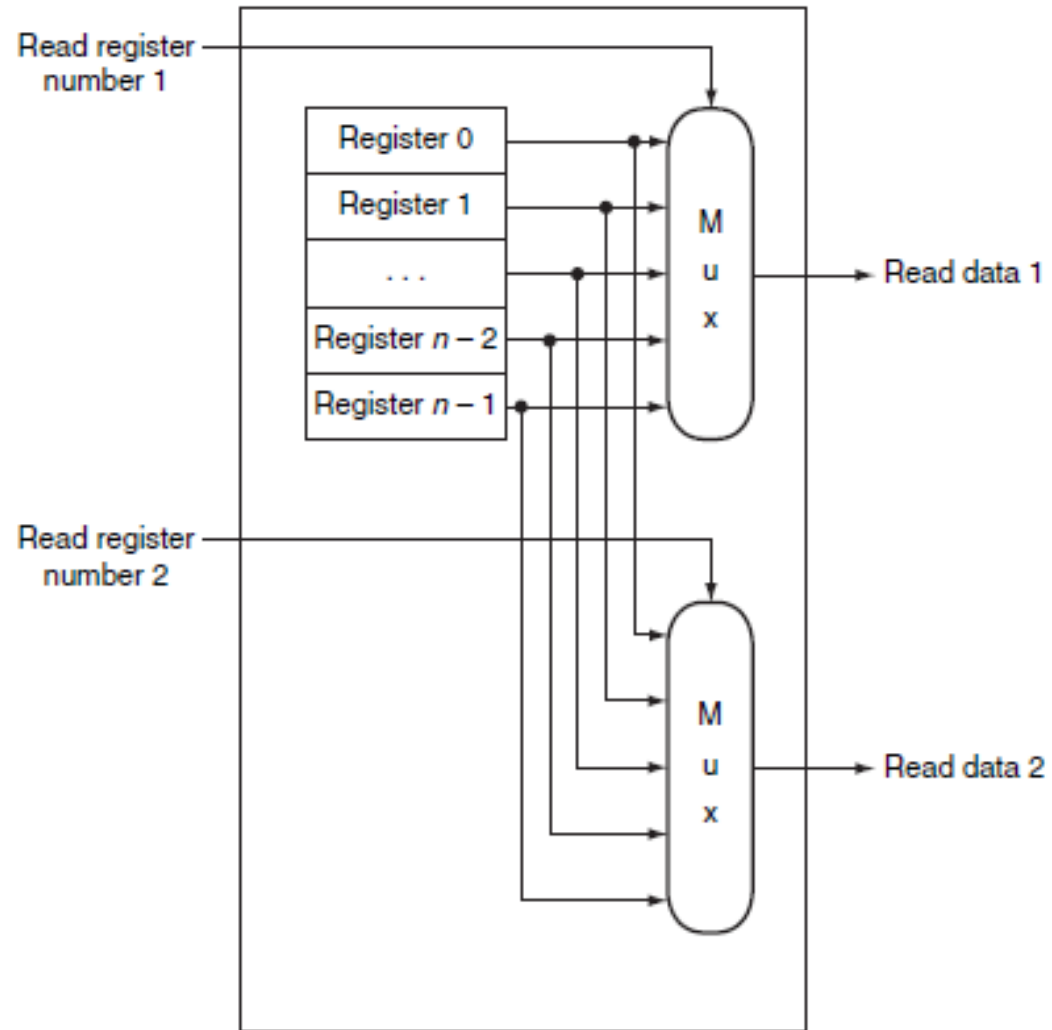
- Capable of holding a single binary value
- Loads a new value when write is enabled
- 4-bit Data Register



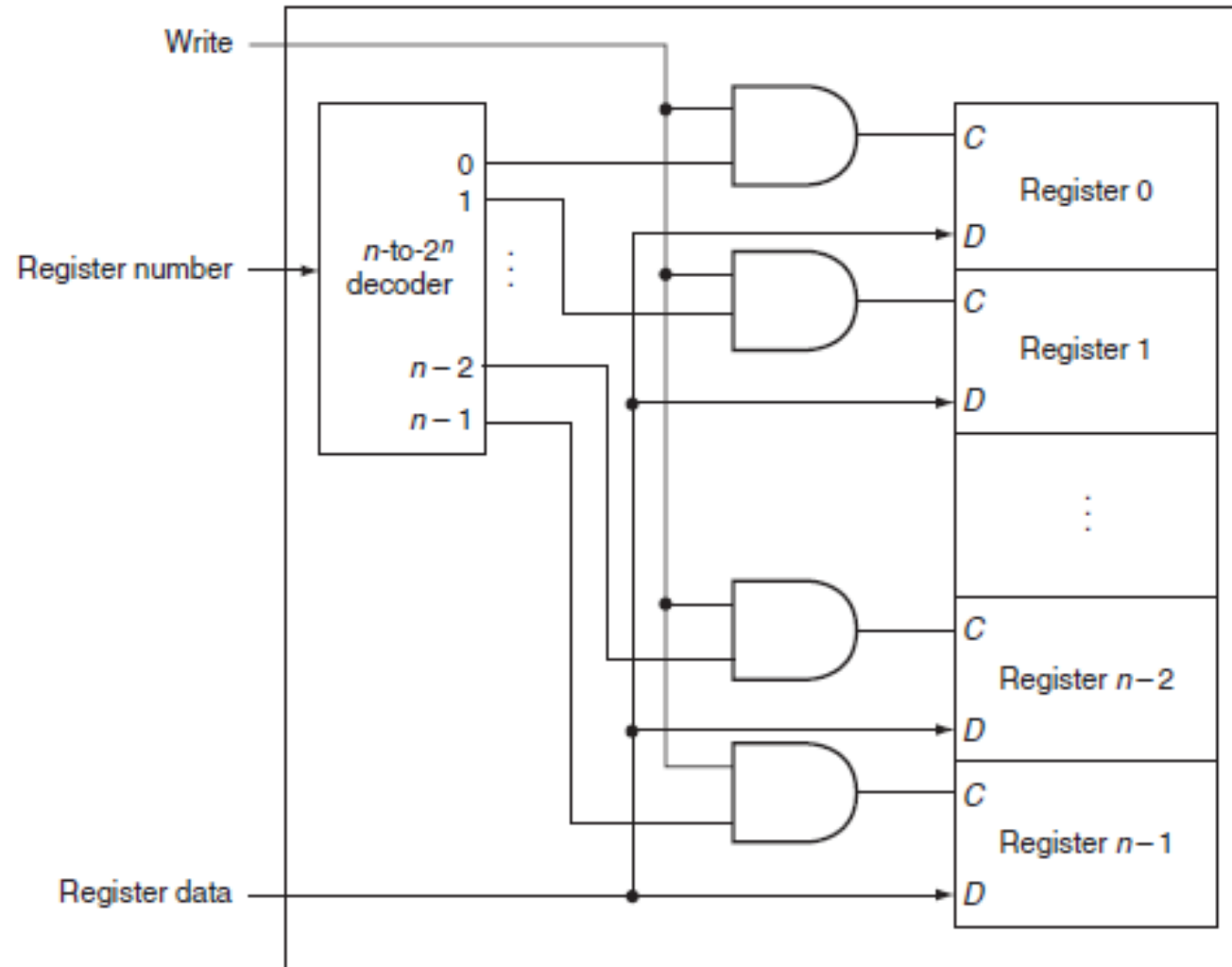
Register Files

- A register file consists of a set of registers that can be read and written by supplying a register number to be accessed.

Register Files: Read



Register Files: Write





MESUT OZDAG, PH.D.
MESUT.OZDAG@UCF.EDU

